



Escuela Técnica Superior de
Ingeniería Informática

Trabajo Complementos de Bases de Datos

DeckBuilder: desarrollo de un gestor de barajas multijuego con C# y MongoDB

Realizado por

**Borrego González, Adolfo
Sánchez Carmona, Germán**

Departamento
Lenguajes y Sistemas Informáticos

Grado
Grado en Ingeniería Informática - Ingeniería del Software

Repositorio del proyecto
github.com/germansanchezz/CBD-GR-24

Índice general

1	Introducción	1
2	Fase 1: Planificación y Diseño	2
2.1.	Ideas y Alcance	2
2.2.	Análisis del Flujo de Usuario y Casos de Uso	2
2.3.	Diseño de la Arquitectura de Datos en MongoDB	3
2.4.	Stack Tecnológico	4
2.5.	Resultado de la fase	4
3	Fase 2: Desarrollo CORE	6
3.1.	Implementación del núcleo funcional	6
3.2.	Desarrollo del backend y consolidación del CRUD	6
3.3.	Desarrollo del frontend y flujo operativo real	8
3.4.	Capacidades multijuego y editor visual de baraja	11
3.5.	Resultados de la fase	12
4	Fase 3: Memoria Técnica y Presentación	13
4.1.	Elaboración de la documentación técnica	13
4.2.	Preparación de la presentación	13
4.3.	Desarrollo del archivo README: Manual usuario y despliegue	13
4.4.	Resultados finales de la fase	14
5	Conclusiones	15
6	Declaración sobre el uso de herramientas de IA	16
	Bibliografía	18

1. Introducción

En los últimos años se ha podido observar un notable crecimiento en la popularidad de los juegos de cartas coleccionables (TCG) como pueden ser Pokémon, Magic: The Gathering y Yu-Gi-Oh! Por esto surge la necesidad de usar una herramienta centralizada que permite la construcción de mazos y gestión de inventario. Este trabajo describe el diseño e implementación de DeckBuilder, una aplicación que busca facilitar e integrar las tareas de los coleccionistas.

El proyecto consiste en el desarrollo de una API REST que permita operaciones CRUD (Create, Read, Update, Delete) sobre barajas e inventarios de cartas, implementado en C# mediante .NET 8 y usando MongoDB como base de datos. La elección de estas tecnologías, más en concreto de MongoDB, permite gestionar datos heterogéneos con estructuras y atributos variables, lo que encaja perfectamente con nuestra propuesta multijuego.

El objetivo principal del proyecto es construir una aplicación capaz registrar cartas obtenidas a través de APIs externas dedicadas [1, 2, 3], de forma que se garantice lo máximo posible la precisión de la información. De este modo, se busca resolver la necesidad de usar múltiples aplicaciones para gestionar las colecciones de distintas franquicias.

El proyecto se estructura en tres fases:

1. Planificación y Diseño
2. Desarrollo e implementación
3. Documentación y Presentación

Como resultado, se ha obtenido una aplicación funcional que destaca por su capacidad de adaptarse a distintos esquemas de datos gracias al uso de bases de datos documentales.

2. Fase 1: Planificación y Diseño

2.1. Ideas y Alcance

En esta primera fase dejamos establecidas las bases del proyecto. Lo primero que hicimos, fue tener una reunión inicial donde definimos el alcance, mediante este, determinamos que para que el sistema fuera útil, debía ser capaz de manejar varios juegos de cartas distintos sin tener que cambiar la base de datos para cada una de las franquicias.

Durante esta primera reunión, contemplamos diversas formas en las que podíamos orientar el proyecto con las tecnologías que habíamos escogido. Tras verificar que podíamos usar APIs externas para la extracción de las cartas, llegamos a la idea de que poder crear barajas personalizadas era lo que más sentido tenía para implementar el CRUD completo que se nos requería.

2.2. Análisis del Flujo de Usuario y Casos de Uso

Para que la aplicación fuera operativa, tuvimos que pensar en un flujo de usuario completo y lógico que proporciona valor y fuera útil para los usuarios, este flujo se documentó con los siguientes pasos:

- **Identificación y Autenticación:** El usuario accede al sistema, lo que permite vincular sus barajas personalizadas a su perfil único almacenado en MongoDB.
- **Consulta y Descubrimiento:** El usuario utiliza el buscador. Este componente realiza peticiones a la API externa de tcgdex.dev [1], devolviendo el listado de cartas candidatas.
- **Persistencia en Colección:** Una vez seleccionada una carta, el sistema CRUD realiza una operación de creación (CREATE) para guardar la referencia y los metadatos específicos.
- **Construcción de Barajas:** El usuario crea un contenedor "Baraja" donde añade y retira cartas. Aquí se aplica la lógica de edición (UPDATE) para modificar cantidades o estados.
- **Ver y Eliminar:** El usuario puede ver sus barajas lo que sería la funcionalidad de (READ) y podría si quisiera eliminarla con la operación de (DELETE).

2.3. Diseño de la Arquitectura de Datos en MongoDB

Debido a nuestro interés como equipo en las bases de datos no relacionales, seleccionamos la tecnología de MongoDB, lo que implicó que la arquitectura fuera un punto clave de la planificación. A diferencia de los modelos SQL, optamos por un esquema basado en documentos, que nos permite una total flexibilidad con los atributos.

Los modelos de datos definidos se muestran en las Figuras 2.1, 2.2 y 2.3.

```
_id: ObjectId('69eddaca2223af44157b5217')
Email: "german@test.com"
PasswordHash: "8D969EEF6ECAD3C29A3A629280E686CF0C3F5D5A86AFF3CA12020C923ADC6C92"
DisplayName: "german"
CreatedAtUtc: 2026-04-26T09:28:42.877+00:00
UpdatedAtUtc: 2026-04-26T09:28:42.877+00:00
```

Figura 2.1: Modelo datos User (MongoDB)

```
_id: ObjectId('69ee2d71627c88bc04b782f5')
Name: "Magic Vampiros"
Description: ""
GameType: "magic"
OwnerUserId: ObjectId('69eddaca2223af44157b5217')
▼ Cards: Array (6)
  ▼ 0: Object
    CardId: "fee330e7-bb67-4379-856f-f99282da6a9f"
    Name: "Vampiro de la Baronia"
    ImageUrl: "https://cards.scryfall.io/normal/front/f/e/fee330e7-bb67-4379-856f-f99..."
    ▼ Properties: Array (2)
      0: "Criatura"
      1: "Negro"
    Quantity: 2
  ▼ 1: Object
    CardId: "810d072f-9140-46ce-9ea5-5c8326a0535a"
    Name: "Vampiro de Refugio Cornisa"
    ImageUrl: "https://cards.scryfall.io/normal/front/8/1/810d072f-9140-46ce-9ea5-5c8..."
    ▼ Properties: Array (3)
      0: "Criatura"
      1: "Negro"
      2: "Blanco"
    Quantity: 2
  ▶ 2: Object
  ▶ 3: Object
  ▶ 4: Object
  ▶ 5: Object
CreatedAtUtc: 2026-04-26T15:21:21.192+00:00
UpdatedAtUtc: 2026-04-26T15:22:08.999+00:00
```

Figura 2.2: Modelo datos Deck (MongoDB)

```

_id: ObjectId('69ee2d83627c88bc04b782f9')
UserId: ObjectId('69eddaca2223af44157b5217')
GameType: "magic"
ExternalCardId: "41189401-8741-4388-aae6-5514600101d1"
Name: "Vampiro de Guul Draz"
ImageUrl: "https://cards.scryfall.io/normal/front/4/1/41189401-8741-4388-aae6-551..."
SetName: "Zendikar"
Rarity: "common"
TypeLine: "Creature — Vampire Rogue"
SearchTags: Array (1)
MainText: "As long as an opponent has 10 or less life, this creature gets +2/+1 a..."
QuantityOwned: 1
QuantityInDecks: 1
AddedAtUtc: 2026-04-26T15:21:39.567+00:00
UpdatedAtUtc: 2026-04-26T15:21:39.769+00:00

```

Figura 2.3: Modelo datos User_Card (MongoDB)

2.4. Stack Tecnológico

El requisito principal del proyecto era construir una API REST que permitiese trabajar con una base de datos NoSQL en MongoDB. Para este proyecto en concreto, teníamos la restricción de que la tecnología a utilizar debía ser C#, por lo que construimos el backend en .NET 8. Este se encarga de la lógica de negocio y exponer la API.

Por otro lado, se decidió incluir un frontend mediante React + Vite + TypeScript. Aunque no era requisito obligatorio, pensamos que facilitaría en gran medida la interacción de los usuarios con la API.

En cuanto a infraestructura, el proyecto se desplegó en servicios cloud:

- Render para backend y frontend [4].
- MongoDB Atlas para alojar la base de datos en entorno gestionado [5].

Por tanto la aplicación mantiene el núcleo requerido en MongoDB y C#, añadiendo una arquitectura web completa que queda preparada para su uso en un entorno real de producción.

2.5. Resultado de la fase

En esta fase hemos podido establecer una buena base para comenzar el desarrollo de las funcionalidades del proyecto. Durante esta etapa han quedado definidos de forma clara los objetivos del sistema, el alcance y el flujo de usuario, asegurándonos de esta forma que la aplicación cubre todas las necesidades esenciales.

Como resultado de esta fase, hemos obtenido una visión clara del funcionamiento que queremos para el sistema, incluyendo la interacción entre el usuario, con la aplicación y con las fuentes externas de datos. Además, se ha

quedado definido el stack tecnológico. Esto nos ha facilitado la transición a la fase de desarrollo, donde hemos podido implementar el sistema de una manera más eficiente y con menor riesgo de errores en la estructura.

3. Fase 2: Desarrollo CORE

3.1. Implementación del núcleo funcional

En esta segunda fase del proyecto nos centramos en pasar todas las ideas y la planificación realizada a un producto software. El objetivo principal ha sido construir una base que permitiera al usuario cubrir el flujo completo de la aplicación, desde el proceso de autenticarse hasta la gestión avanzada de las barajas y las colecciones.

Para asegurar la calidad de la aplicación, la implementación la realizamos siguiendo un enfoque modular e incremental. Este enfoque se basó en finalizar módulos concretos, lo cual nos permitió verificar de una forma constante todas las funcionalidades implementadas, antes de continuar con las siguientes, minimizando los riesgos con el producto.

3.2. Desarrollo del backend y consolidación del CRUD

En el backend, desarrollamos una API utilizando el entorno de .NET 8 diseñada para soportar todas las operaciones sobre las barajas de usuarios. Durante este desarrollo, se priorizó la separación de responsabilidades mediante el uso de clases DTO y de validaciones independientes, lo que nos permite mantener el código de una manera sencilla.

Sobre este sistema creamos un sistema CRUD completo para la entidad de barajas: creación (CREATE), lectura (READ), actualización (UPDATE) y borrado (DELETE). La lógica de actualización fue algo más difícil, ya que debíamos incluir la persistencia dinámica de las cartas dentro de cada uno de los mazos con sus respectivas reglas de dominio específicas, como son el caso de las copias máximas permitidas o el tamaño total de la baraja.

Para garantizar que los datos externos que se utilizan fueran estables, mejoramos las peticiones con los proveedores de cartas mediante el desarrollo de un mejor cliente HTTP. Realizamos pequeños ajustes en los algoritmos de búsqueda para asegurar que las respuestas obtenidas desde las distintas fuentes fueran consistentes y estuvieran listas para ser usadas y procesadas.

Los ejemplos de modelos, controlador y validaciones se muestran en los Listados 3.1, 3.2 y 3.3.

```
1 public sealed class Deck
2 {
3     public string Name { get; set; } = string.Empty;
4     public string GameType { get; set; } = DeckGameTypes.Pokemon;
```



```
5 public List<DeckCard> Cards { get; set; } = new();
6 public DateTime UpdatedAtUtc { get; set; } = DateTime.UtcNow;
7 }
```

Extracto de código 3.1: Modelos de Deck y de User (C#)

```
1 [HttpPost]
2 public async Task<IActionResult> CreateDeck([FromBody] CreateDeckRequest
   request)
3 {
4     if (!DeckRequestValidator.TryValidateCreate(request, out var error))
5         return BadRequest(new { message = error });
6
7     await GetDecksCollection().InsertOneAsync(deck);
8     return Created($"/api/decks/{deck.Id}", deck);
9 }
```

Extracto de código 3.2: Ejemplo controlador (C#)

```
1 if (totalCards > MaxCardsPerDeck)
2 {
3     errorMessage = $"Una baraja no puede superar {MaxCardsPerDeck} cartas
   .";
4     return false;
5 }
6
7 if (copiesByName[normalizedName] > maxCopiesPerName)
8 {
9     errorMessage = $"No puedes tener mas de {maxCopiesPerName} copias con
   el mismo nombre.";
10    return false;
11 }
```

Extracto de código 3.3: Ejemplo validaciones (C#)

La API desarrollada sigue un diseño REST, con recursos como usuarios, barajas y cartas. Esta organización nos ha permitido una separación clara de responsabilidades y nos facilita el poder escalar el sistema.

Los endpoints implementados cubren tanto la autenticación como la gestión completa de barajas y la colección de cartas del usuario.

Autenticación

- POST /api/auth/register: registro nuevos usuarios
- POST /api/auth/login: autenticación de usuarios existentes

Gestión de barajas (Decks)

- GET /api/decks: obtención de todas las barajas del usuario autenticado
- GET /api/decks/{deckId}: consulta detallada de baraja específica
- POST /api/decks: creación de nueva baraja
- PUT /api/decks/{deckId}: actualización de una baraja, incluyendo modificar cartas
- DELETE /api/decks/{deckId}: eliminación de una baraja

Gestión de colección de cartas (User Cards)

- GET api/user-cards: obtención de colección de cartas del usuario (README, detalles)
- GET api/user-cards/stats: obtención de estadísticas de colección (README, detalles)
- POST api/user-cards: inserción de nuevas cartas en colección
- POST api/user-cards/{userCardId}/quantity: ajuste incremental de copias carta
- DELETE api/user-cards/{userCardId}: eliminación carta de colección

Búsqueda de cartas

- GET api/cards/search: búsqueda de cartas fuentes externas [1, 2, 3]

Este conjunto de endpoints forma la base del sistema CRUD, no solo sobre barajas sino también sobre la colección personal del usuario. Los endpoints de estadísticas y búsquedas externas aportan un valor añadido al sistema.

El funcionamiento detallado y su configuración, se encuentran explicados en el archivo README del repositorio, donde se tienen en cuenta aspectos como la lógica de interacción, las validaciones y la comunicación con la base de datos.

3.3. Desarrollo del frontend y flujo operativo real

Mientras trabajabamos en el backend, también estuvimos desarrollando la interfaz de usuario que comenzó siendo un prototipo muy sencillo hasta llegar a un flujo completo y dinámico. Implementamos un sistema para la gestión de sesiones con las posibilidades de registro e inicio de sesión, garantizando que el usuario únicamente acceda a su colección almacenada en MongoDB una vez este se ha identificado.

Una vez completamos la validación de acceso, comenzamos a desarrollar las pantallas de colección y de detalle para cada barajas. Estas interfaces permiten a los usuarios interactuar en tiempo real con todo su contenido, editando su mazo y persistiendo con los cambios de forma inmediata con la API. Este avance en el proyecto supuso una refactorización del cliente por algunas áreas de responsabilidad para luego facilitar la escalabilidad futura.

Para finalizar el desarrollo de la interfaz, añadimos la funcionalidad de borrado definitivo de barajas y se añadieron diseños responsive a la aplicación. Estas mejoras ayudan a que la experiencia del usuario fuera la mejor posible.

Las pantallas principales de la interfaz se muestran en las Figuras 3.1, 3.2, 3.3, 3.4 y 3.5.

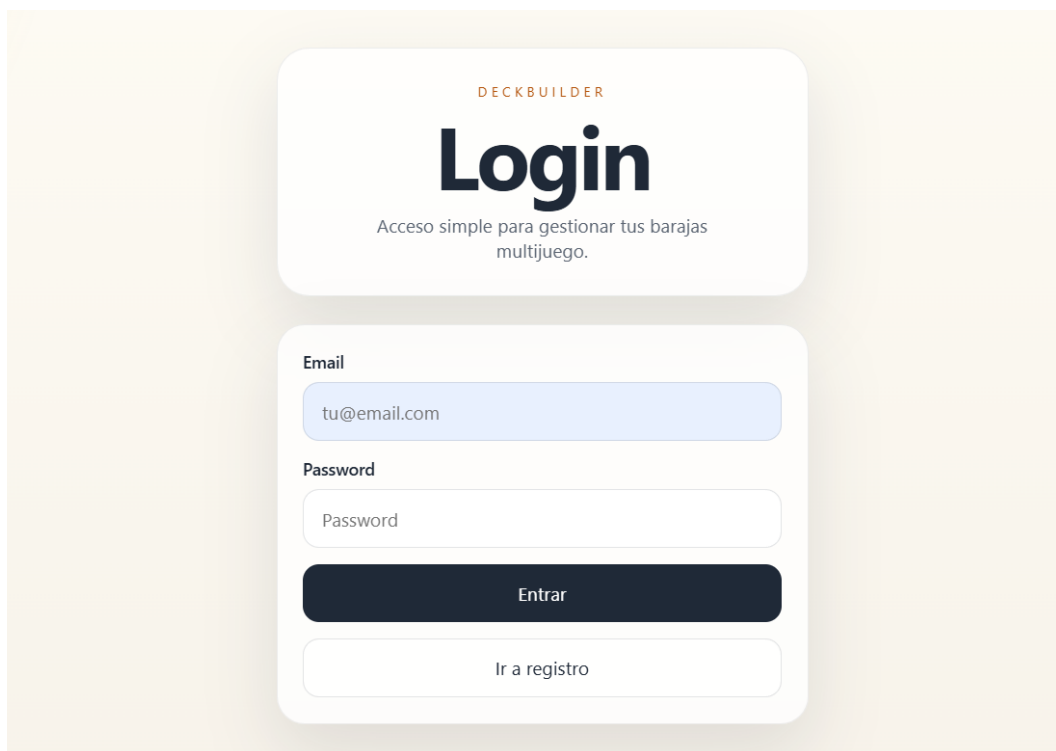


Figura 3.1: Pantalla de Login/Registro

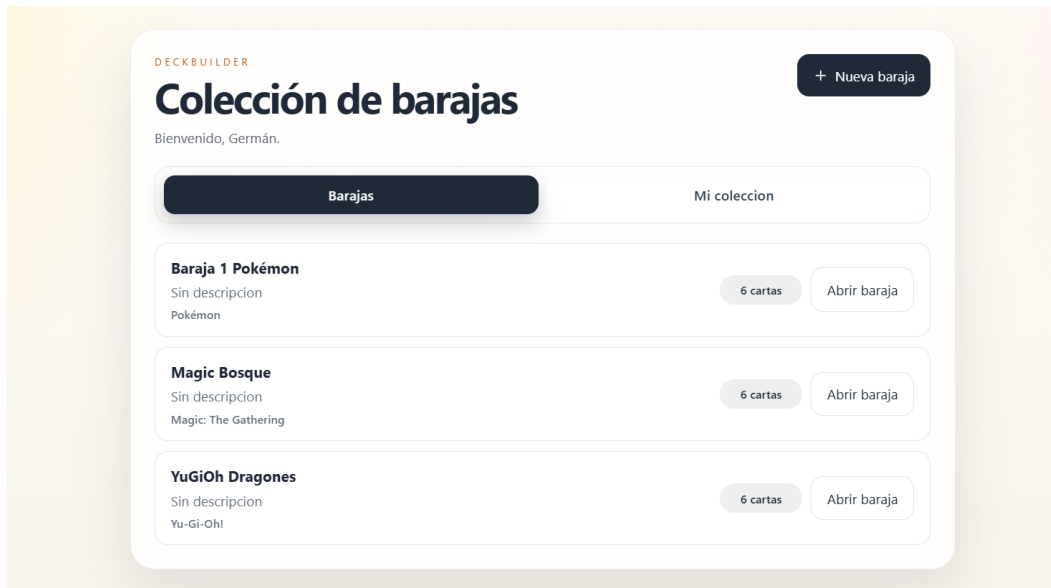


Figura 3.2: Pantalla de Colección de barajas

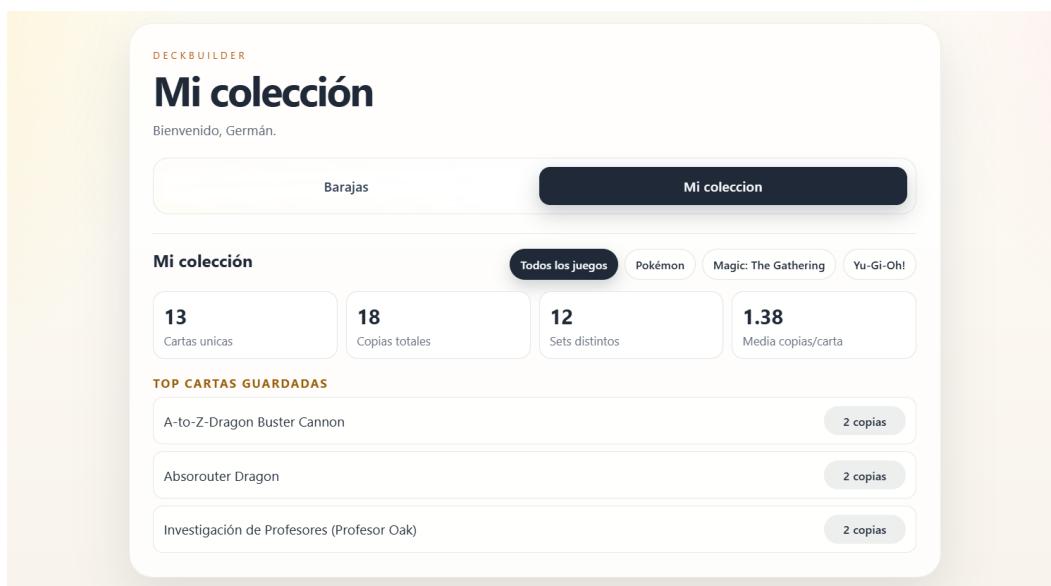


Figura 3.3: Pantalla de Mi colección

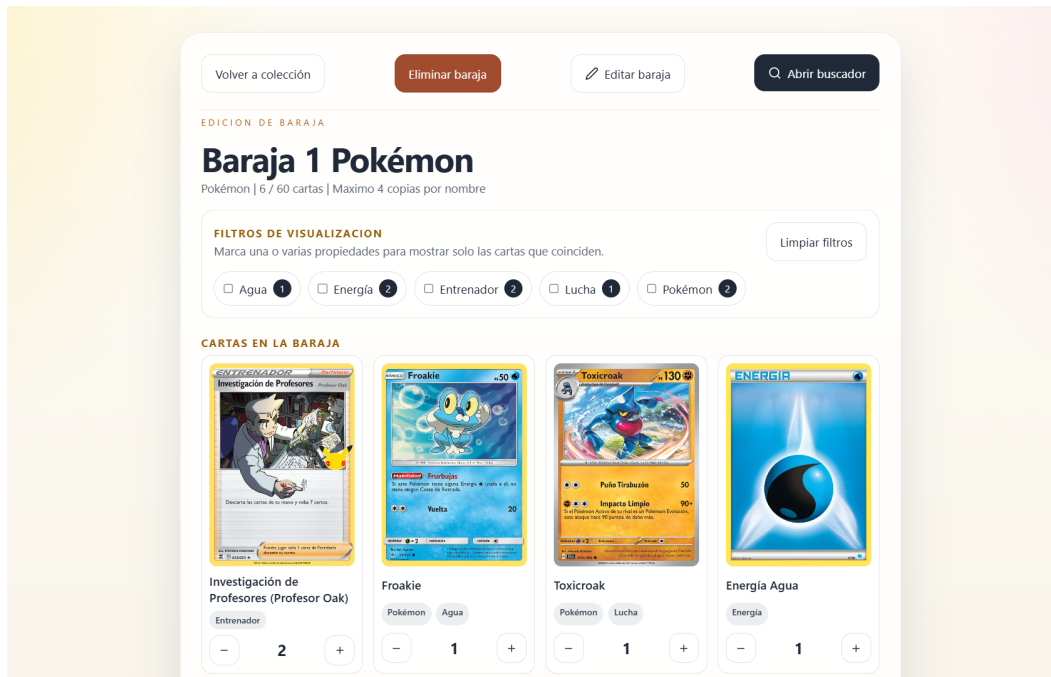


Figura 3.4: Pantalla de Mi baraja

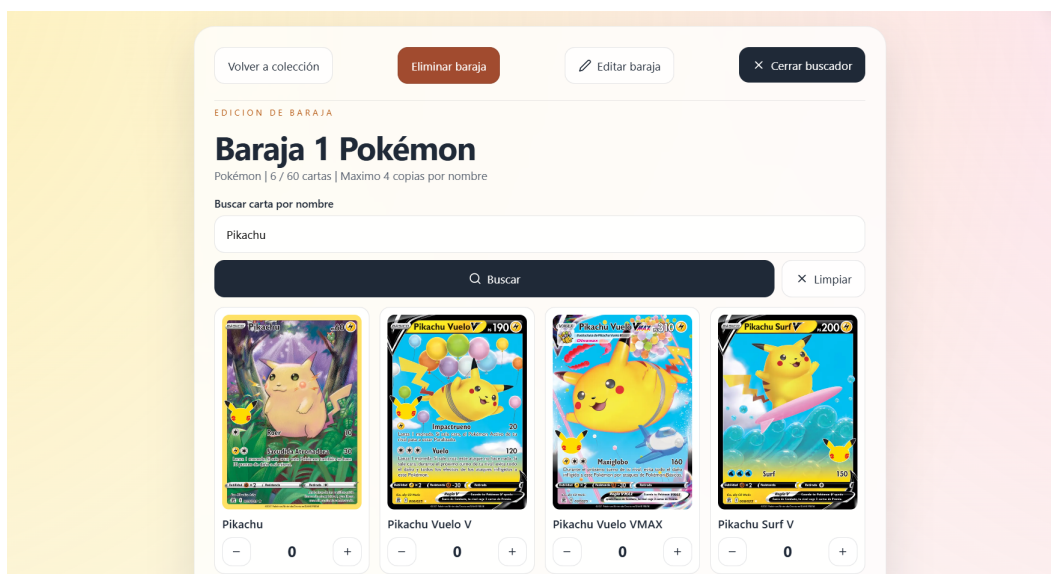


Figura 3.5: Pantalla de Buscador de cartas

3.4. Capacidades multijuego y editor visual de baraja

Una de las partes más importantes fue transformar la plataforma en un sistema capaz de integrar barajas procedentes de diferentes juegos. En la aplicación hemos integrado el tipo de juego “gametype” (Pokémon, Magic o Yu-Gi-Oh!) como un atributo dentro de la base de datos de MongoDB, permitiendo que el sistema se adapte a lo que el usuario seleccione cuando cree una nueva baraja.

Gracias a este diseño multijuego pudimos desarrollar un endpoint de búsqueda unificado capaz de consultar en múltiples fuentes externas [1, 2, 3]. Con esta implementación permitimos que el usuario pueda buscar las cartas de cada una de las franquicias utilizando el mismo buscador, simplificando así la complejidad.

Para finalizar, construimos el editor de barajas. Esta herramienta permite que los usuarios añadan o retiren cartas mediante controles directos, mientras el sistema se encarga de forma automática de las reglas de validación de cada uno de los juegos, garantizado que el mazo resultante sea legal dentro de cada uno de los juegos.

El atributo `GameType` se muestra en el Listado 3.4.

```
1 public static class DeckGameTypes
2 {
3     public const string Pokemon = "pokemon";
4     public const string Magic = "magic";
5     public const string Yugioh = "yugioh";
6 }
7
8 var cards = normalizedGameType switch
9 {
10     DeckGameTypes.Pokemon => await SearchPokemonCardsAsync(httpClient,
11     query, cancellationToken),
12     DeckGameTypes.Magic => await SearchMagicCardsAsync(httpClient, query,
13     cancellationToken),
14     DeckGameTypes.Yugioh => await SearchYugiohCardsAsync(httpClient,
15     query, cancellationToken),
16     _ => []
17 };
18
```

Extracto de código 3.4: Atributo `GameType` (MongoDB)

3.5. Resultados de la fase

La fase 2 la terminamos de manera exitosa con un núcleo funcional completo que integra la autenticación, la vinculación de datos por usuarios, el CRUD completo y el motor multijuego. El sistema ha demostrado un correcto funcionamiento permitiéndonos la construcción de mazos completos de forma muy clara y sencilla.

Desde el punto de vista de la arquitectura, la aplicación queda preparada para añadir nuevos juegos de manera sencilla sin tener que reestructurar la base de datos para cada franquicia. De esta manera hemos terminado el desarrollo de toda la interfaz de la aplicación así como de la lógica de la misma, dejando así una buena base para el proyecto.

4. Fase 3: Memoria Técnica y Presentación

4.1. Elaboración de la documentación técnica

Durante la tercera y última fase del proyecto nos centramos en documentar todas las fases del proyecto, con el objetivo de cumplir los requisitos de evaluación de la asignatura. Nos centramos en redactar la memoria, documento que no solo describe el sistema final, sino que justifica las decisiones de diseño que fueron tomadas por el grupo a lo largo de todo el desarrollo, como pueden ser la elección del stack tecnológico o la implementación de las reglas de negocio.

Elaborar la documentación nos ha permitido asegurar que todo lo que hemos realizado en las fases anteriores queda registrado. La memoria funciona también como un soporte que demuestra el aprendizaje en el uso de C# y MongoDB, detallando cómo hemos incluido la variabilidad en la estructura de los datos en un sistema CRUD completamente funcional.

4.2. Preparación de la presentación

Otra parte fundamental de esta fase ha sido la preparación de la presentación. Este proceso ha consistido en resumir los aspectos más relevantes del desarrollo para su exposición. Diseñamos materiales visuales que destacan los puntos fuertes del proyecto, como la integración de APIs externas y la flexibilidad de la base de datos documental.

4.3. Desarrollo del archivo README: Manual usuario y despliegue

Como cierre de desarrollo del proyecto, hemos elaborado un archivo README dentro del repositorio del código fuente. Este archivo cumple dos funciones importantes: servir como manual de usuario y como guía de despliegue e instalación. En este documento quedan explicados paso a paso todas las instrucciones necesarias para que cualquier usuario que quiera utilizar la aplicación pueda configurarlo en su entorno local y una guía de uso que describe el flujo completo de uso.

4.4. Resultados finales de la fase

Las conclusiones de esta última fase marcan el fin del desarrollo de DeckBuilder. Con la memoria técnica finalizada, la presentación preparada y el archivo README en el repositorio, el proyecto queda listo para su entrega final. El resultado es una aplicación completamente funcional, apoyada por una documentación mediante la que se puede comprender todo lo realizado durante el desarrollo de DeckBuilder.

5. Conclusiones

Tras finalizar el desarrollo de Deckbuilder hemos logrado cumplir el objetivo principal que era diseñar e implementar una aplicación funcional que utiliza las tecnologías de C# y MongoDB.

Una de las características más importantes del proyecto ha sido el uso de una base de datos documental. Gracias al uso de MongoDB hemos podido gestionar de la mejor forma la variabilidad de los datos entre los distintos juegos de cartas que hemos usado, evitando de esta forma las limitaciones de los modelos relaciones que conocemos.

Viéndolo desde un punto de vista técnico, el desarrollo de un CRUD completo, junto con el frontend interactivo, nos ha permitido cubrir todo el flujo del usuario dentro de la aplicación, desde la autenticación hasta la gestión completa de las barajas y colecciones.

En conclusión, el proyecto nos ha permitido aplicar de forma práctica los conceptos estudiados en la asignatura, especialmente en el uso de bases de datos NoSQL, consolidando de esta forma una buena base con estas nuevas tecnologías.

6. Declaración sobre el uso de herramientas de IA

Durante el desarrollo de este proyecto se han utilizado herramientas de inteligencia artificial como apoyo al trabajo realizado por los autores. En concreto, se ha recurrido a ChatGPT, Gemini y GitHub Copilot en distintas fases del proyecto, siempre como soporte y nunca como sustituto del criterio técnico o del trabajo de desarrollo, redacción y maquetación.

ChatGPT y Gemini se han empleado principalmente para:

- resolver dudas sobre el uso de tecnologías como MongoDB y C#;
- apoyar la redacción de partes de la memoria técnica, mejorando su claridad, estructura y redacción;
- proponer ideas para mejorar la lógica interna de la aplicación;
- generar ejemplos de peticiones y respuestas para entender mejor el funcionamiento de la API.

GitHub Copilot se ha utilizado durante el desarrollo del código y también en tareas de documentación. Su uso ha sido especialmente útil para:

- autocompletar funciones en C# y TypeScript;
- sugerir implementaciones en la lógica del backend y del frontend;
- acelerar la implementación del frontend, que no era un requisito inicial estricto del proyecto y se desarrolló como ampliación funcional;
- asistir en la redacción y revisión de la memoria técnica;
- facilitar el paso de una versión inicial redactada en Google Docs a su posterior maquetación en LaTeX.

Ejemplos de prompts utilizados:

- “Quiero que revises si la memoria cumple ahora mismo todos los requisitos que se piden en las instrucciones de entrega y en las cuestiones sobre los trabajos. Este es el README que hace de manual de despliegue y manual de usuario: README.md.”
- “Quiero utilizar componentes de react-icons para los botones de la app, por ejemplo el de cerrar sesión o el más y menos de las cartas.”
- “Quiero hacer un proyecto CRUD conectando C# (.NET) con MongoDB. Necesito ideas de proyecto que tengan sentido; el proyecto lo realizamos entre dos personas.”

Algunas conversaciones con ChatGPT:

- <https://chatgpt.com/share/69ee4684-5020-83eb-9fa2-355e1040c50c>
- <https://chatgpt.com/share/69ecf754-aeb0-8397-b190-0e320f49cae1>

En todo caso, el uso de estas herramientas se ha limitado a tareas de apoyo, revisión y sugerencia. Las decisiones finales de diseño, implementación, redacción y maquetación han sido tomadas por los autores del proyecto.

Bibliografía

- [1] TCGdex. Tcgdex api documentation, 2026. URL <https://tcgdex.dev/>.
- [2] Scryfall. Scryfall api documentation, 2026. URL <https://scryfall.com/docs/api>.
- [3] YGOPRODeck. Ygoprodeck api guide, 2026. URL <https://ygoprodeck.com/api-guide/>.
- [4] Render. Render documentation, 2026. URL <https://render.com/docs>.
- [5] MongoDB. Mongoddb atlas documentation, 2026. URL <https://www.mongodb.com/docs/atlas/>.